

Введение в программирование на языке C++

Компиляция и интерпретация

- **Интерпретатор** – это программа, которая получает исходную программу и по мере распознавания конструкций входного языка реализует действия, описываемые этими конструкциями.
- **Транслятор** – принимает на вход исходную программу и порождает программу, записанную на объектном языке программирования (объектный код).

- **Компилятор** транслирует текст всей программы в модуль на машинном языке, затем программа переписывается в оперативную память и лишь после этого исполняется процессором компьютера. Полученный модуль можно многократно запускать на выполнение.
- Язык C++ является ***компилируемым*** языком программирования высокого уровня.

Обзор компиляторов языка C++

- *GNU GCC*
- ***Microsoft Visual C++***
- *Intel C++ compiler*

Обзор сред программирования на языке C++

- **Интегрированная среда разработки** (*IDE – Integrated Development Environment*), как правило, содержит собственный текстовый редактор, имеет связь с компилятором или встроенный компилятор, позволяя скомпилировать и запустить программу по одному клику мыши, а также еще много вспомогательных возможностей, например, отладчик и автодополнение вводимого текста.

Среды программирования на языке C++

- *Microsoft Visual Studio*
- *Code::Blocks*
- *NetBeans IDE*
- *Qt Creator*
- *Dev-C++*
- *Xcode*

Типичная структура программы

```
#include "stdio.h"

int main()
{

    return 0;

}
```

- Директива **#include** используется для подключения других файлов в код.
- В контексте операционной системы, каждое приложение должно иметь **точку входа**. Такой точкой входа является функция **main()**. Именно с нее начинается исполнение приложения после его загрузки в память вычислительной системы.

Алфавит языка C++

Строчные и прописные буквы латинского алфавита	a, b, c, ..., y, z A, B, C, ..., Y, Z
Арабские цифры	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Знак нижнего подчеркивания	_
Специальные символы	{ } ; , \ () " ' # [] + - * / % . : ? < = > ! & ~ ^
Пробельные символы	Пробел, символ табуляции, символ перевода строки

- ***Лексема*** – минимальная единица языка, имеющая смысл для транслятора.
- К лексемам относят:
 - идентификаторы;
 - ключевые (зарезервированные) слова;
 - знаки операций;
 - литералы;
 - разделители (точка, скобки, запятая, пробельные символы).

Идентификаторы

- **Идентификатор** – это имя программного объекта.
- Идентификаторы могут состоять из заглавных и строчных латинских букв, цифр и знака подчеркивания.
- **При этом на первом месте не может стоять цифра!**
- Язык C++ является регистрозависимым, то есть идентификаторы **A** и **a** – **различные**.

Примеры идентификаторов



_ABC,
A10c
this_name_is_too_long
x_365



7xB,
A Func
Abc_87F&_7

- **Ключевые слова** – это predetermined идентификаторы, которые имеют специальное значение для компилятора языка C++.
- **Имена объектов программы не могут совпадать с ключевыми словами.**
- К ключевым словам относятся, например, наименования типов данных.

- **Знак операции** – символ или группа символов, задающих некоторую операцию.
- **Литерал** – это неизменяемая и неадресуемая величина.
- Литерал хранится в памяти компьютера, но нельзя узнать его адрес.

Основные типы данных в C++

- **bool** — логический тип данных. Хранит одно из двух значений: **true** (истина) или **false** (ложь).
- СИМВОЛЬНЫЕ ТИПЫ,
- ЦЕЛОЧИСЛЕННЫЕ ТИПЫ ДАННЫХ,
- ТИПЫ ДАННЫХ С ПЛАВАЮЩЕЙ ТОЧКОЙ.

СИМВОЛЬНЫЕ ТИПЫ

- Для представления ASCII-символов используется тип **char** (8 бит).
- Для широкого представления символов используется тип **wchar_t**.
- Для представления символов в кодировке UTF16 – **char16_t**, а для представления символов в UTF32 – **char32_t**.

Целочисленные типы

- Базовым целочисленным типом является тип **int**.
- Чаще всего под переменную данного типа отводится 32 бита, старший бит резервируется под знак (0 – неотрицательные числа, 1 – отрицательные).

модификаторы **int**:

- **signed** – целевой тип будет иметь знаковое представление (по умолчанию, если не представлен ни один из вариантов);
- **unsigned** – целевой тип будет иметь беззнаковое представление, в этом случае знаковый бит не выделяется;
- **short** – короткое целое, под переменную резервируется 16 бит;
- **long** – длинное целое, под переменную резервируется 32 бита;
- **long long** позволяет создавать 64-битные переменные

Типы с плавающей точкой

- Для представления вещественных чисел в языке C++ используются типы с плавающей точкой, определяемые стандартом IEEE754.
- **float** – тип с плавающей точкой одинарной точности. Обычно 32битный тип;
- **double** – тип с плавающей точкой двойной точности. Обычно 64битный тип;
- **long double** – тип с плавающей точкой повышенной точности.

Переменные

- **Переменная** – именованная область памяти, в которой хранятся данные определенного типа.
- У переменной есть имя и значение.
- Имя служит для обращения к области памяти, в которой хранится значение.
- Во время выполнения программы значение переменной можно изменять.
- Перед использованием любая переменная должна быть описана.

Объявление переменных

```
int a; // объявление переменной a целого типа.  
float b; // объявление переменной b типа данных с плавающей запятой.  
double c = 14.2; // инициализация переменной типа double.  
char d = 's'; // инициализация переменной типа char.  
bool k = true; // инициализация логической переменной k.
```

Операции

- **Операция** (*operator*) – конструкция в языке программирования, обозначающая действие над одним или несколькими объектами.
- По количеству принимаемых аргументов (*операндов*) операции делят на:
унарные (один аргумент),
бинарные (два аргумента)
и тернарные.

Для операций существует три варианта синтаксиса:

- **префиксная** (польская) нотация: **+ a b**;
- **инфиксная** нотация: **a + b**;
- **постфиксная** (обратная польская) нотация: **a b +**.
- Для бинарных и тернарных операций в языке C++ используется инфиксная нотация, для большинства унарных операций – префиксная.

Операция присваивания

- Полная форма:

`<имя_переменной> = <выражение>;`

- Сокращенная форма:

`<имя_переменной><операция>=<выражение>;`

`x *= 5;                    //x = x * 5`

`s += 7;                    //s = s + 7`

`y /= x + 3;                //y = y / (x + 3)`

Арифметические операции

- Арифметические операции применимы к числовым типам данных.
- Их можно подразделить на *унарные* и *бинарные*.

Унарные операции

- К унарным относятся операции *инкремента* ($++$) и *декремента* ($--$).
- При помощи данных операций значение целочисленной переменной можно увеличить ($++$) или уменьшить ($--$) на 1.
- Операции могут быть *постфиксные* ($i++$) и *префиксные* ($++i$).

```
/*операция*/ ++value; /*эквивалентна операции*/ value += 1;  
/*операция*/ --value; /*эквивалентна операции*/ value -= 1;  
/*операция*/ ++value; /*эквивалентна операции*/ value = value +1;  
/*операция*/ --value; /*эквивалентна операции*/ value = value - 1;
```

Бинарные операции

- К *бинарным* относятся стандартные арифметические операции (+, −, *, /) и операция получения остатка (%).
- В случае если оба операнда – целочисленные, результат операции деления будет также целочисленным.
- Если получается дробный результат, то происходит округление к ближайшему меньшему по модулю целому числу.
- Для того чтобы получить вещественный результат, необходимо привести один из операндов к вещественному типу.

Логические операции

- Логические операции применимы **только в логических выражениях** и служат для составления сложных условий, требующих более одной операции отношения.
- В случае если на месте операнда стоит число, то производится неявное преобразование типа.
- Любое число, отличное от нуля, интерпретируется как **true**, а ноль — как **false**.

Операторы отношения и логические операторы языка C++

Операторы отношений	Значение
==	Равно
!=	Не равно
>	Больше
<	Меньше
>=	Больше или равно
<=	Меньше или равно
Логические операторы	Значение
&&	И
	ИЛИ
!	НЕ

Поразрядные операции

- Поразрядные (побитовые) операции можно производить с любыми целочисленными переменными и константами.
- **Эти действия не применимы к вещественным переменным.**
- Результат поразрядной операции – целое число.
- Поразрядные операции выполняются над двоичным представлением целого числа.
- Операция применяется по отдельности к каждому биту числа.

Поразрядные операции

~ – поразрядное отрицание – каждый бит числа инвертируется;

& – поразрядное И – бит результата равен 1 тогда и только тогда, когда соответствующие биты обоих операндов равны 1;

| – поразрядное ИЛИ – бит результата равен 1 тогда и только тогда, когда соответствующий бит хотя бы одного из операндов равен 1;

^ – поразрядное исключающее ИЛИ – бит результата равен 1 тогда и только тогда, когда соответствующий бит ровно одного из операндов равен 1.

Операции сдвига

- К поразрядным операциям относятся операции сдвига, которые выполняются над двоичным представлением числа, но в данном случае биты числа рассматриваются не по отдельности, а в совокупности.

>> – сдвиг вправо

- Сдвиг двоичного представления первого операнда вправо на число разрядов, заданное вторым операндом.
Если первый операнд беззнакового типа, то освободившиеся биты заполняются нулями, в противном случае – знаковым битом числа.
- Операция аналогична целочисленному делению на степень двойки.

<< – СДВИГ ВЛЕВО

- Сдвиг двоичного представления первого операнда влево на число разрядов, заданное вторым операндом, освободившиеся биты заполняются нулями.
- Операция аналогична умножению на степень двойки без учета переполнения.

Примеры использования СДВИГОВ

1. Найти $45 \gg 3$

00101|101 $\xrightarrow{\text{сдвиг на 3 вправо}}$ 000|00101

2. Найти $45 \ll 3$

001|01101 $\xrightarrow{\text{сдвиг на 3 влево}}$ 01101|000

Получилось $45 \ll 3 = 104$.

Операция sizeof

- Операция получения размера предназначена для вычисления размера объекта или типа в байтах.

sizeof <выражение>

sizeof (<ТИП>)

Операции ввода/вывода в C++

Традиционный ввод-вывод

```
#include <cstdio>

int main()
{
    printf("Hello, world!\n");
}
```

Ввод-вывод с помощью потоков STL

```
#include <iostream>

using namespace std;

int main() {
    cout << "Hello, world!\n";
}
```

Основные алгоритмические конструкции

- Следование
- Ветвление
- Цикл

Условный оператор

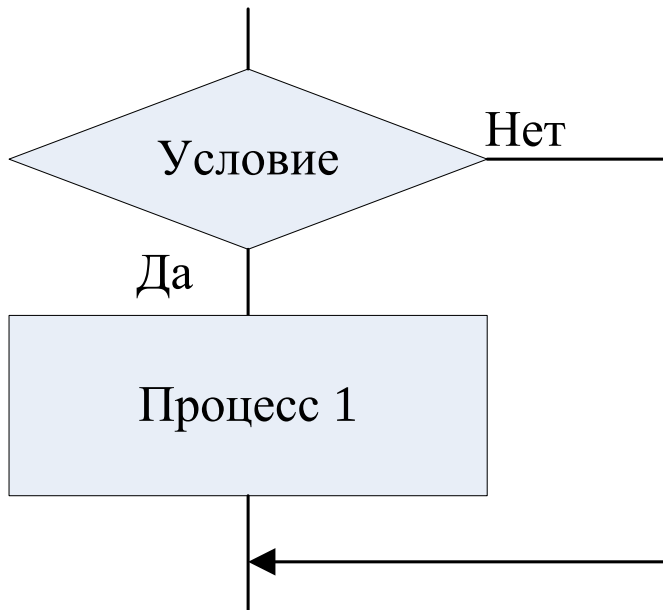
- **Неполная форма**

if (условие) { действие 1 }

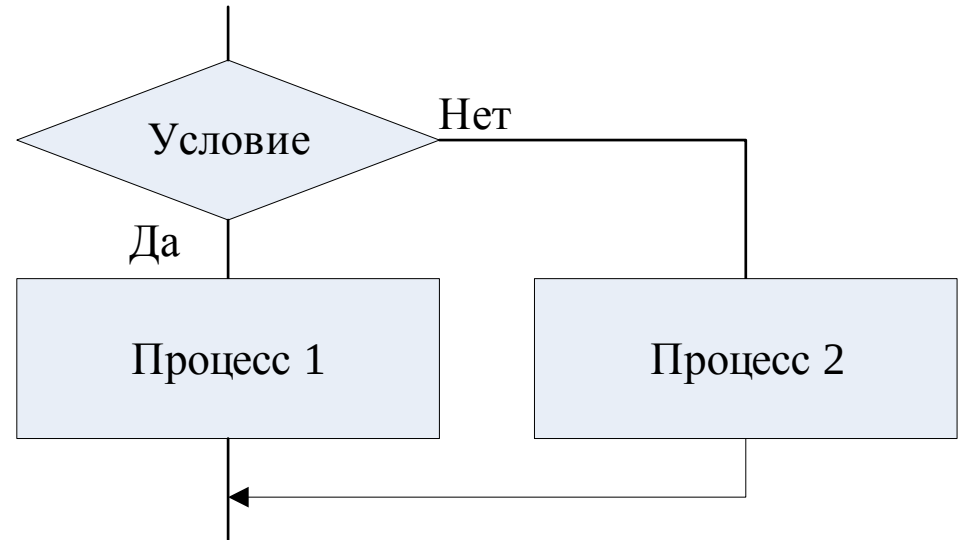
- **Полная форма**

if (условие) { действие 1 }
 else { действие 2 }

- Неполная форма



- Полная форма



Оператор выбора (переключатель)

Оператор **switch** используется для выбора одного из нескольких направлений дальнейшего хода программы.

switch (переменная-селектор)

case значение 1 : действие 1; **break**;

case значение 2 : действие 2; **break**;

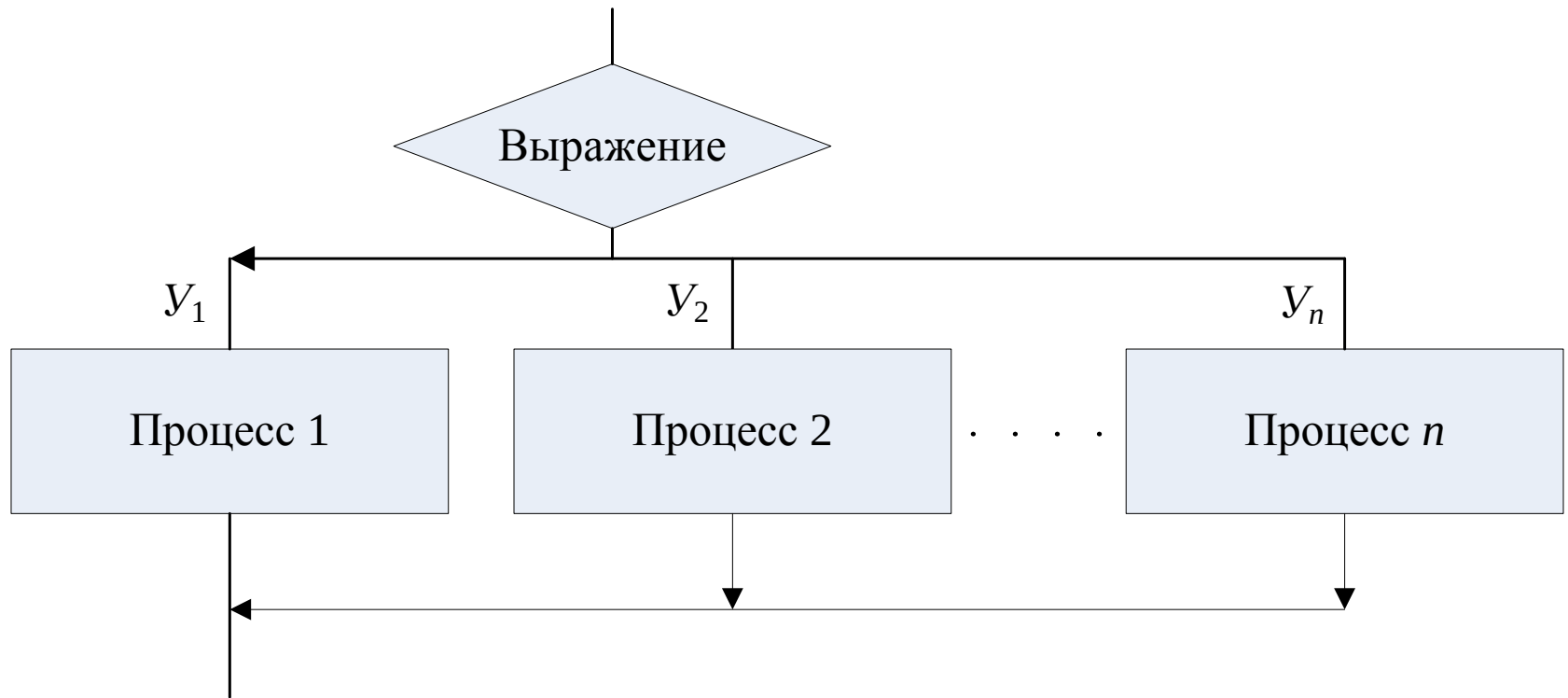
...

case значение n: действие n; **break**;

default: действие n+1;

- Оператор **switch** проверяет, совпадает ли значение выражения с одним из значений, приведенных ниже **констант**.
- При совпадении выполняются операторы, стоящие после совпавшей константы.

Множественное ветвление



Пример:

```
char c;
int t= scanf_s("%c", &c,1);
switch (c)
{
case 'П': printf_s("Превосходно\n"); break;
case 'В': printf_s("Выше Ожидаемого\n"); break;
case 'У': printf_s("Удовлетворительно\n"); break;
case 'С': printf_s("Слабо\n"); break;
case 'О': printf_s("Отвратительно\n"); break;
case 'Т': printf_s("Троль\n"); break;
default: printf_s("Не верно введена оценка\n");
break;
}
```

Операция условного перехода ?:

**имя_переменной =
условие ? выражение_1 : выражение_2;**

- Если **условие** истинно, то **имени_переменной** присваивается результат **выражения_1**,
иначе – **выражения_2**.

Пример

- Найти наибольшее из двух чисел:

max = a > b ? a : b;

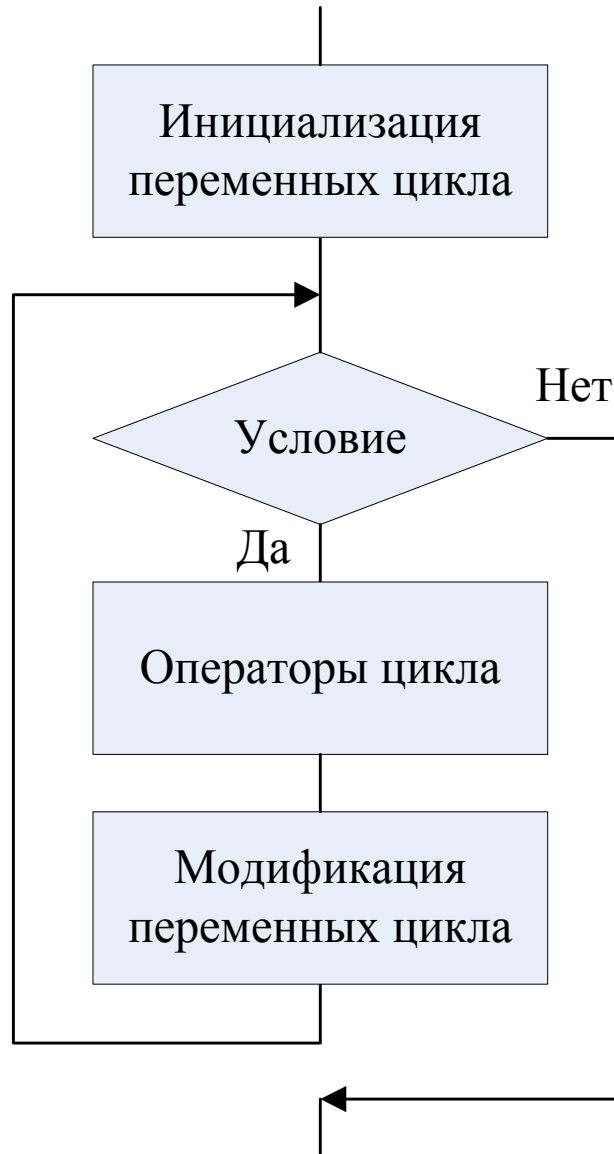
Цикл

- Циклический вычислительный процесс характеризуется повторением одних и тех же вычислений над некоторым набором данных.
- Числом повторений цикла управляет специальная переменная, называемая его **счетчиком** или **управляющей переменной цикла**. На счетчик накладывается условие, определяющее, до каких пор следует выполнять цикл.
- Повторяемый блок вычислений называют **телом цикла**.
- В теле цикла должно быть обеспечено изменение значения счетчика, чтобы он мог завершиться.

Цикл с предусловием

- В цикле с предусловием сначала проверяется условие, затем, в зависимости от того, истинно оно или ложно, либо выполняется тело цикла, либо следует переход к оператору, следующему за телом цикла.
- После завершения тела цикла управление вновь передается на проверку условия.
- Естественно, предполагается, что в теле цикла было обеспечено некоторое изменение входящих в условие переменных – в противном случае произойдет *зацикливание* и программа "зависнет".

Схема цикла с предусловием



Оператор цикла с предусловием

while (условие)

{

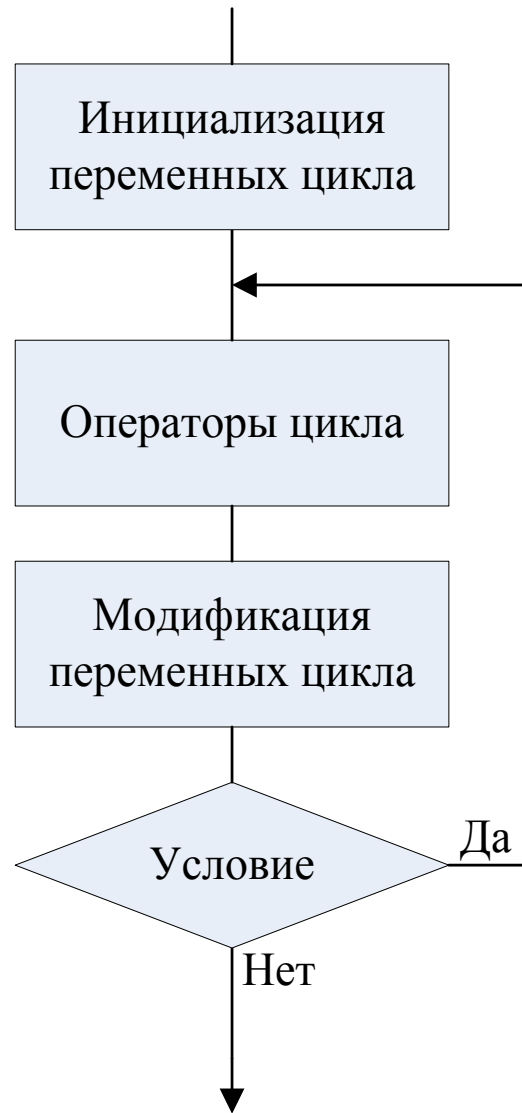
 тело цикла;

}

Цикл с постусловием

- Для цикла с постусловием сначала выполняется тело цикла, затем управление передается на проверку условия.
- В зависимости от истинности или ложности условия, тело цикла выполняется повторно или же происходит переход к оператору, следующему за телом цикла.

Схема цикла с постусловием



Оператор цикла с постусловием

do

{

 тело цикла

}

while (условие);

При выполнении цикла **do ... while** один проход цикла будет выполнен независимо от условия.

Цикл с параметром

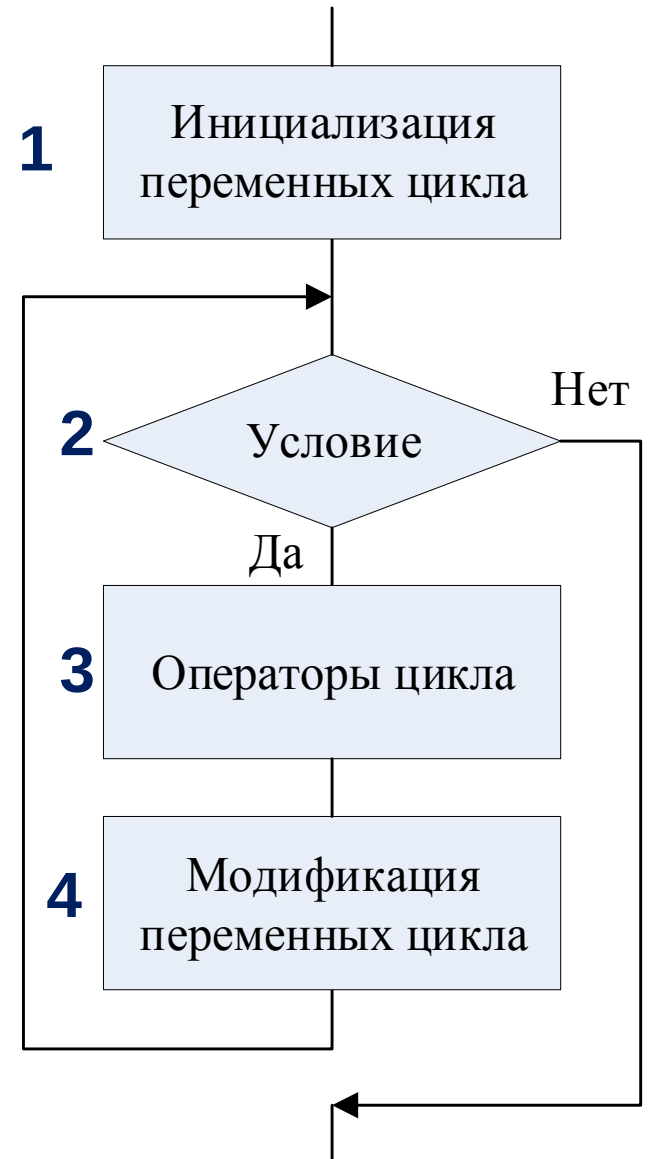
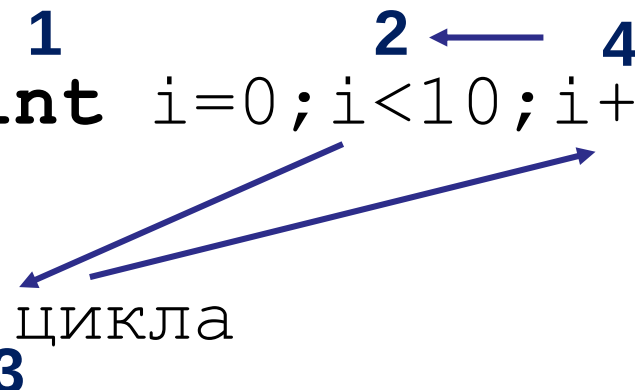
```
for (действие до начала цикла;  
    условие продолжения цикла;  
    действия в конце каждой итерации цикла) {  
    инструкция цикла;  
    инструкция цикла 2;  
    инструкция цикла N;  
}
```

```
for (счетчик = значение; счетчик < значение; шаг цикла) {  
    тело цикла;  
}
```

Схема цикла с параметром

```
1  
for (int 2 i=0; i<10; i++) 4  
{  
  тело цикла  
}
```

3



Пример: накопление суммы

1. Для подсчета каждой суммы описать по одной переменной того же типа, что суммируемые данные;
2. До цикла переменной-сумме присвоить начальное значение 0;
3. В теле цикла, если очередной элемент данных t отвечает условию суммирования, сумма накапливается оператором вида $s=s+t$;

Задача: сосчитать сумму натуральных чисел от 1 до 1000.

```
#include <iostream>
using namespace std;

int main()
{
    int i; // счетчик цикла
    int sum = 0; // сумма чисел от 1 до 1000.
    setlocale(0, "");
    for (i = 1; i <= 1000; i++) // задаем начальное значение 1, конечное 1000 и задаем шаг
        цикла - 1.
    {
        sum = sum + i;
    }
    cout << "Сумма чисел от 1 до 1000 = " << sum << endl;
    return 0;
}
```


Накопление произведения

1. Для подсчета произведения описать переменную того же типа, что перемножаемые данные;
2. До цикла переменной-произведению присвоить начальное значение 1;
3. В теле цикла, если очередной элемент данных t отвечает условию перемножения, произведение накапливается оператором вида $p = p * t$;